

Agentes Reflexivos (Reflex Agents) — Análisis completo de la Sesión 10

Fuente base: *Agentes IA — Reflex Agents* — Módulo 4 (Agentes Cognitivos), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora.

Complementado con: investigación propia sobre la taxonomía clásica de agentes de Russell & Norvig (*Artificial Intelligence: A Modern Approach*), la clasificación práctica de IBM (*Simple Reflex, Model-Based Reflex, Goal-Based, Utility-Based, Learning Agents*), la formalización con MDP/POMDP, fundamentos de *Reinforcement Learning* (Sutton & Barto), patrones modernos de agentes LLM como ReAct, Reflexion, Self-Refine y Tree of Thoughts, y la implementación moderna con LangChain 1.x / LangGraph 1.x.

Propósito de este documento: las Sesiones 8 y 9 definieron la arquitectura general del agente y luego abrieron la capa de memoria/contexto. Esta sesión baja un nivel más: clasifica el **programa de decisión** del agente. La pregunta ya no es solo "qué componentes tiene", sino **cómo decide una acción**: por regla inmediata, por modelo interno, por meta, por utilidad o por aprendizaje.

0. Dónde se ubica esta sesión — de memoria contextual a decisión reflexiva

Sesión 8 – Arquitectura de Agentes:

Agente = comunicación + contexto + entorno + autonomía + criticidad
└ Se define QUÉ componentes existen.

Sesión 9 – Memoria Contextual:

Context Window + Short/Long Term Memory + Store/Checkpointing
└ Se define QUÉ recuerda el agente y cómo entra al contexto.

Sesión 10 – Agentes Reflexivos:

Percepto → Estado interno → Meta/Utilidad/Aprendizaje → Acción
└ Se define CÓMO el agente selecciona una acción.

La primera diapositiva de la sesión recapitula explícitamente el modelo de contexto de la Sesión 9:

Contexto Real del Problema

- └ Contexto Documentado del Problema
 - └ Contexto del Usuario
 - └ Context Window

Ese recordatorio no es decorativo. En esta sesión, el *Context Window* se conecta con la toma de decisiones: para un agente basado en modelo, objetivos, utilidad o aprendizaje, el contexto disponible determina qué estado cree estar observando, qué metas entiende, qué recompensas puede optimizar y qué feedback puede incorporar.

1. Objetivos y agenda de la sesión

Objetivos declarados en el PDF:

1. Entender los tipos de Agentes Reflexivos.
2. Comprender sus diversas estrategias de implementación.

Agenda del Bloque A:

#	Tema	Qué se trabaja
1	Tipos de Agentes Reflexivos	Se introduce la progresión: Simple Reflex y Model-Based Reflex
2	Simple Reflex Agent	Reglas estrictas condición-acción, sin memoria ni feedback
3	Model-Based Reflex Agent	Estado interno del mundo que cambia con percepciones
4	Lab	Aterrizar el proyecto propio a Simple Reflex o Model-Based Reflex; implementación Python

Agenda del Bloque B:

#	Tema	Qué se trabaja
1	Conversation Memory Buffer en LangChain	Aunque la agenda lo nombra así, el contenido del bloque se orienta a tipos de decisión más avanzados
2	Goal-Based Reflex Agent	Estado + simulación de futuros estados para lograr una meta
3	Utility-Based Reflex Agent	Estado + simulación + ranking/recompensa para maximizar utilidad

#	Tema	Qué se trabaja
4	Learning Reflex Agent (RLEF)	Aprendizaje por experiencia mediante feedback del ambiente
5	Lab final	Actualizar el <i>Profile Card</i> del proyecto con memoria y tipo(s) de agentes reflexivos; implementación Python

Tarea final indicada en la página 22 del PDF:

Actualizar el *Profile Card* del proyecto acorde al uso de memoria y tipo(s) de agentes reflexivos. Deadline: 22-07. Implementación en Python conforme a los ejemplos.

2. La escalera visual de la sesión — cinco tipos de agente

La página 6 del PDF presenta una progresión visual de cinco niveles:

- 0. Simple Reflex Agent
- 1. Model-Based Reflex Agent
- 2. Goal-Based Reflex Agent
- 3. Utility-Based Reflex Agent
- 4. Learning Reflex Agent

La numeración de la diapositiva arranca en 0, lo cual es pedagógicamente útil: el **Simple Reflex Agent** es el punto mínimo de agencia computable. Hay percepción y acción, pero no hay memoria, planificación, evaluación de utilidad ni aprendizaje. A partir de ahí, cada nivel añade una capacidad:

Nivel	Tipo	Capacidad nueva frente al anterior
0	Simple Reflex	Mapea percepción actual a acción mediante reglas
1	Model-Based Reflex	Mantiene un estado/modelo interno del mundo
2	Goal-Based	Simula o busca estados futuros para alcanzar una meta
3	Utility-Based	Evalúa alternativas con una función de utilidad/recompensa
4	Learning	Ajusta su política con feedback del ambiente

Lectura arquitectónica: estos no son necesariamente "frameworks" distintos. Son patrones de decisión que pueden implementarse con código clásico, con LangChain/LangGraph, con un LLM usando herramientas, con un sistema multiagente, o

incluso sin LLM. La tecnología concreta es secundaria; lo central es **qué información usa el agente para decidir**.

3. Ancla académica — la taxonomía de Russell & Norvig

La clasificación de la sesión coincide con la taxonomía clásica de *Artificial Intelligence: A Modern Approach* (Russell & Norvig), donde la estructura de agentes incluye:

Tipo en AIMA	Correspondencia en la sesión	Idea central
Simple reflex agents	Simple Reflex Agent	Reglas condición-acción sobre el percepto actual
Model-based reflex agents	Model-Based Reflex Agent	Estado interno para entornos parcialmente observables
Goal-based agents	Goal-Based Reflex Agent	Búsqueda/planificación hacia un objetivo
Utility-based agents	Utility-Based Reflex Agent	Maximización de utilidad esperada
Learning agents	Learning Reflex Agent	Mejora del desempeño mediante experiencia y feedback

En la notación formal de AIMA, un agente implementa una función:

$$f : P^* \rightarrow A$$

donde P^* es la secuencia de percepciones históricas y A es el conjunto de acciones posibles. La diferencia entre los cinco tipos está en **cuánto de esa historia usa realmente el programa del agente**:

Tipo	Uso real de P^*
Simple Reflex	Usa solo el último percepto p_t
Model-Based Reflex	Usa percepciones pasadas para mantener un estado interno $\hat{s}_t$
Goal-Based	Usa $\hat{s}_t$ y un objetivo G para elegir una ruta de acciones
Utility-Based	Usa $\hat{s}_t$, objetivos/criterios y una función $U(s)$ para comparar futuros
Learning	Actualiza su política $\pi(a \mid s)$ con feedback/recompensas

Distinción terminológica importante: en esta sesión, "reflexivo" se usa en el sentido clásico de *reflex agent* (un agente que decide como respuesta a percepciones). No significa

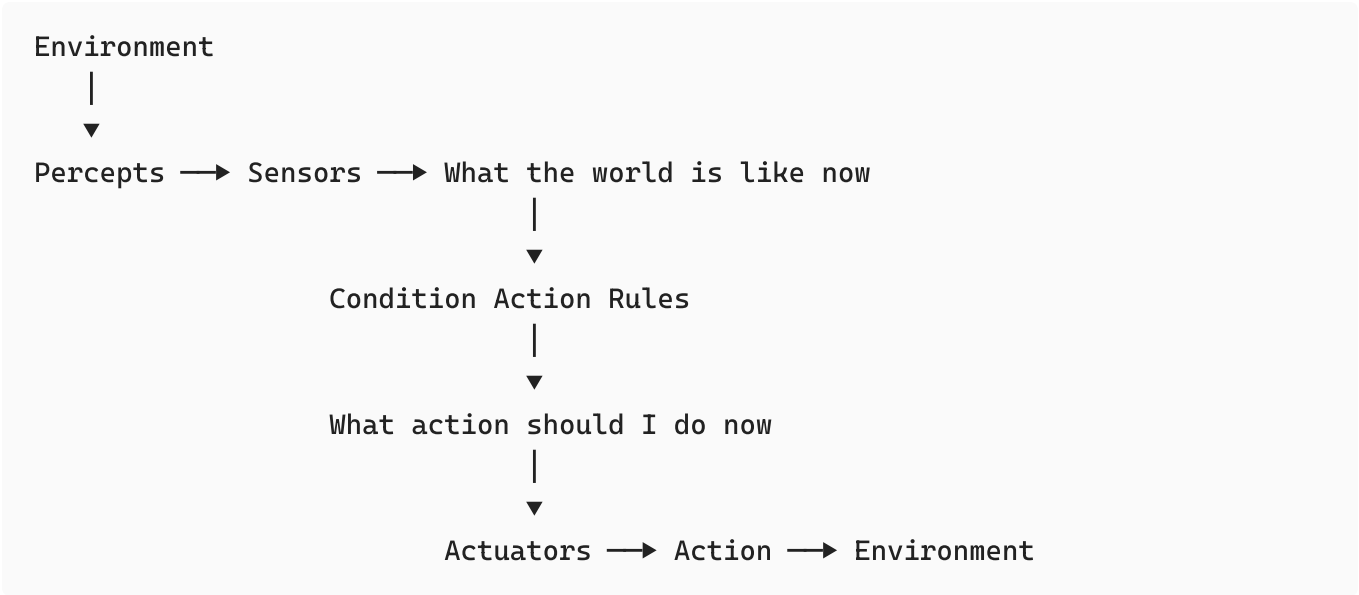
necesariamente "auto-reflexivo" en el sentido moderno de LLMs que se critican a sí mismos. Esa segunda familia aparece en la investigación complementaria como **Reflexion** y **Self-Refine**, pero no es idéntica al término usado por AIMA/IBM.

4. Simple Reflex Agent — reglas condición-acción sin memoria

La página 7 define el Simple Reflex Agent con dos ideas fuertes:

- Reglas = Condicionales estrictas mapeadas a acciones
- No hay feedback

El diagrama de la página 7 muestra el flujo clásico:



4.1 Qué lo caracteriza

Dimensión	Simple Reflex Agent
Entrada	Percepción actual
Estado interno	No tiene
Memoria	No
Planificación	No
Función de utilidad	No
Aprendizaje	No

Dimensión	Simple Reflex Agent
Decisión	Regla si condición entonces acción
Entorno ideal	Totalmente observable, estable, predecible
Latencia	Muy baja

La clave es que el agente **no pregunta qué pasó antes** ni **qué pasará después**. Solo evalúa el estado actual contra una tabla de reglas.

4.2 Pseudocódigo de la diapositiva

La página 8 lo expresa como una función simple:

```
función SimpleReflexAgent(percepto):
    reglas = {
        "temperatura < 18": "encender calefacción",
        "temperatura > 24": "apagar calefacción",
        "presión < 30": "activar bomba",
        "presión > 70": "desactivar bomba"
    }

    para cada condición en reglas:
        si cumple(percepto, condición):
            acción = reglas[condición]
            ejecutar(acción)
            retornar acción

    retornar "sin acción"
```

4.3 Formalización mínima

Un Simple Reflex Agent implementa:

$$a_t = R(p_t)$$

donde:

- p_t = percepto actual,
- R = conjunto de reglas condición-acción,
- a_t = acción seleccionada.

No aparece p_{t-1} , ni un estado interno, ni una meta explícita, ni una función de utilidad.

4.4 Ejemplos típicos

Dominio	Regla
Termostato	Si temperatura < umbral, encender calefacción
Seguridad industrial	Si sensor detecta intrusión, activar alarma
Inventario básico	Si stock < mínimo, emitir alerta
IVR/RPA	Si usuario marca opción 2, transferir a cobranzas
Moderación básica	Si texto contiene patrón prohibido, bloquear

4.5 Fortalezas

- Es rápido y barato.
- Es fácil de auditar.
- Tiene comportamiento predecible.
- Es adecuado para acciones de baja ambigüedad.
- Puede ser más seguro que un LLM cuando las reglas son completas y el dominio es cerrado.

4.6 Limitaciones

- No aprende de errores.
- Repite errores si la regla es insuficiente.
- Falla en entornos parcialmente observables.
- No resuelve conflictos entre reglas salvo que se programe prioridad explícita.
- No sabe posponer una acción para recolectar más información.
- No planifica hacia una meta.

Regla práctica: si puedes expresar el caso como una tabla estable de reglas verificables, no necesitas un agente LLM. Un Simple Reflex Agent determinista suele ser más barato, más trazable y más controlable.

5. Implementación del laboratorio — 01_simple_reflex_agent.py

El repositorio `agents26_m4s10-main` implementa el Simple Reflex Agent con LangChain 1.x y Ollama:

```
01_simple_reflex_agent.py
```

5.1 Caso implementado

El caso simula un sistema de climatización para zonas de almacén:

Zona	Objetivo
zona_A_congelados	-18.0 °C
zona_B_refrigerados	4.0 °C
zona_C_ambiente	21.0 °C

La percepción se obtiene con una función `_leer_sensor(zona)` que simula una lectura instantánea con ruido:

```
deriva = random.uniform(-3.5, 3.5)
return round(objetivo + deriva, 1)
```

5.2 Tools

El agente recibe dos herramientas:

Tool	Rol
<code>sensor_temperatura(zona)</code>	Percibe la temperatura actual de una zona
<code>actuador_climatizacion(zona, accion)</code>	Ejecuta <code>encender_frio</code> , <code>encender_calor</code> o <code>apagar</code>

5.3 System prompt como tabla de reglas

La arquitectura se concentra en el `system_prompt` :

```
1. Lee la temperatura de la zona con la tool `sensor_temperatura`.
2. Si temperatura > objetivo + 2°C -> encender_frio
3. Si temperatura < objetivo - 2°C -> encender_calor
4. En cualquier otro caso -> apagar
5. Responde en una sola línea confirmando la acción tomada.
```

5.4 Decisión técnica clave

El script crea el agente **sin checkpointer ni estado persistente**:

```
agent = create_agent(  
    model="ollama:llama3.2",  
    tools=[sensor_temperatura, actuador_climatizacion],  
    system_prompt=REGLAS,  
)
```

Esto es coherente con la definición: cada `invoke()` es un episodio independiente. Si el agente apagó la climatización hace un turno, no lo recuerda ni lo necesita recordar.

5.5 Observación crítica

Aunque el ejemplo usa un LLM con LangChain, el patrón Simple Reflex **no requiere** un LLM. De hecho, para producción, las reglas de control físico o industrial suelen implementarse como código determinista, y el LLM puede quedar solo como interfaz explicativa:

```
Sensor → Regla determinista → Actuador  
                                |  
                                └─ LLM opcional: explica al usuario por qué se tomó la  
                                acción
```

6. Model-Based Reflex Agent — reglas con estado interno

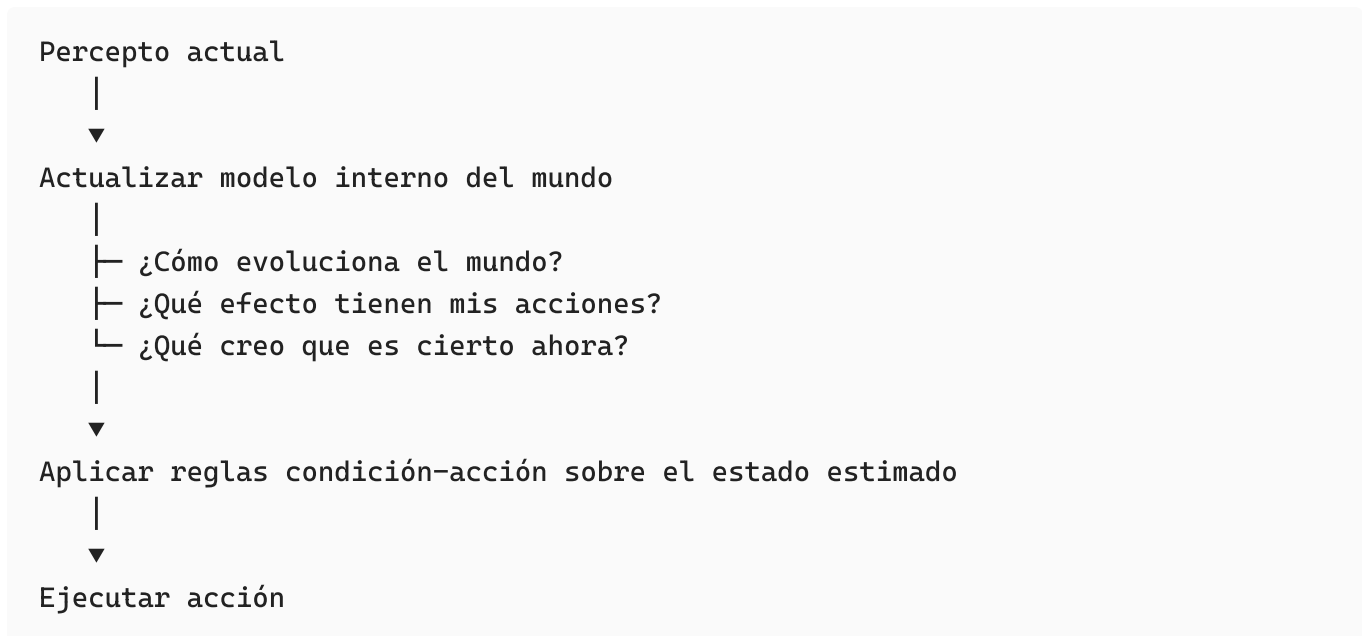
La página 9 agrega la pieza que falta al Simple Reflex Agent:

```
State = Modelo interno del mundo (entorno o ambiente)  
        y cómo cambia sobre acciones  
  
State cambia conforme percepciones
```

El diagrama conserva sensores, reglas y actuadores, pero añade tres bloques:

Bloque	Significado
State	Estado interno actual estimado
How the world evolves	Modelo de dinámica del entorno
What my actions do	Modelo de efectos de las acciones

6.1 Flujo conceptual



6.2 Qué cambia frente al Simple Reflex Agent

Dimensión	Simple Reflex	Model-Based Reflex
Observación	Solo percepto actual	Percepto actual + estado interno
Memoria	No	Sí
Entorno ideal	Totalmente observable	Parcialmente observable
Decisión	Regla sobre percepción	Regla sobre estado estimado
Riesgo principal	No recuerda nada	Puede tener un modelo interno equivocado

El Model-Based Reflex Agent sigue siendo **reactivo**: no planifica una secuencia futura completa. Pero reacciona mejor porque ya no depende solo de lo que ve en este instante.

6.3 Formalización

Un agente basado en modelo mantiene:

$$\hat{s}_t = U(\hat{s}_{t-1}, a_{t-1}, p_t)$$

y luego decide:

$$a_t = R(\hat{s}_t)$$

donde:

- $\hat{s}_t$ = estado interno estimado,
- U = función de actualización del estado,
- a_{t-1} = acción previa,

- p_t = nuevo percepto,
- R = reglas condición-acción.

6.4 Relación con POMDP

En términos de investigación, este tipo de agente aparece cuando el entorno es **parcialmente observable**. En un POMDP (*Partially Observable Markov Decision Process*), el agente no ve el estado real s_t directamente; recibe observaciones o_t que son pistas incompletas. Por eso mantiene una creencia o estado estimado:

$$b_t(s) = P(s_t = s \mid o_{1:t}, a_{1:t-1})$$

El ejemplo del robot explorador del laboratorio es una versión discreta y simple de esta idea: el mapa conocido es el *belief state* del robot.

6.5 Ejemplos típicos

Dominio	Estado interno
Aspiradora robótica	Mapa de zonas limpias, obstáculos y ubicación
Robot de almacén	Celdas visitadas, obstáculos, posición
Chatbot operativo	Caso actual, campos ya recolectados, estado de trámite
Agente de monitoreo	Estado estimado de servicio, incidentes abiertos
Sistema de inventario	Stock estimado, movimientos recientes, discrepancias

7. Implementación del laboratorio — 02_model_based_reflex_agent.py

El laboratorio implementa un robot que explora un almacén en una grilla:

```
02_model_based_reflex_agent.py
```

7.1 Entorno simulado

El almacén se define como matriz:

```
WAREHOUSE_GRID = [
    [0, 0, 0, 0, 1, 0, 0, 0],
    [0, 1, 1, 0, 1, 0, 1, 0],
```

```
...  
]
```

Convención:

Valor	Significado
0	Pasillo libre
1	Obstáculo

Las acciones posibles son:

```
DIRECCIONES = {  
    "norte": (0, -1),  
    "sur": (0, 1),  
    "este": (1, 0),  
    "oeste": (-1, 0),  
}
```

7.2 Estado interno del agente

El script extiende `AgentState`:

```
class EstadoRobot(AgentState):  
    posicion_actual: Annotated[tuple[int, int], _ultimo_valor]  
    mapa_conocido: Annotated[dict[str, str], _fusionar_mapas]
```

Este es el corazón del Model-Based Reflex:

Campo	Función
<code>posicion_actual</code>	Dónde cree estar el robot
<code>mapa_conocido</code>	Qué celdas ya sabe que son libres u obstáculos

7.3 Reducers de LangGraph

El código incluye dos *reducers*:

Reducer	Problema que resuelve
<code>_ultimo_valor</code>	Si hay varias escrituras de posición en un paso, se queda con la última

Reducer	Problema que resuelve
<code>_fusionar_mapas</code>	Si hay varias actualizaciones del mapa, las combina

Esto es una decisión técnica importante. LangGraph trata el estado como canales que pueden recibir actualizaciones durante el grafo. Si más de una tool intenta escribir el mismo campo en el mismo paso, el reducer define cómo fusionar esas escrituras. Sin reducer, se puede producir un error tipo `InvalidUpdateError`.

7.4 Tool que actúa y actualiza memoria

La tool principal es `mover_robot`. Tiene tres responsabilidades:

1. Validar dirección.
2. Intentar mover el robot.
3. Actualizar `posicion_actual` y `mapa_conocido`.

El retorno usa `Command(update=...)`, que permite a una tool modificar el estado del grafo:

```
return Command(update={
    "posicion_actual": posicion_final,
    "mapa_conocido": mapa,
    "messages": [ToolMessage(contenido, tool_call_id=tool_call_id)],
})
```

7.5 Dynamic prompt como memoria activa

El script usa `@dynamic_prompt`:

```
@dynamic_prompt
def prompt_con_modelo_interno(request: ModelRequest) -> str:
    mapa = request.state.get("mapa_conocido", {})
    pos = request.state.get("posicion_actual", (0, 0))
    ...
```

Esto inyecta el estado interno en el prompt en cada paso:

```
Tu posición actual es {pos}.
Tu modelo interno del mundo indica: {resumen_mapa}.
No intentes moverte hacia una celda ya marcada como obstáculo.
```

Conexión con Sesión 9: esto es un ejemplo concreto de *System Prompt dinámico por variables*. El estado no vive como texto fijo: se calcula en runtime y entra al Context Window

cuando el modelo debe decidir.

7.6 Qué enseña este script

Concepto de sesión	Implementación
Estado interno	EstadoRobot
Memoria de corto plazo	Estado del grafo durante la ejecución
Percepción	Resultado de intentar moverse
Modelo del mundo	mapa_conocido
Acción	mover_robot(direccion)
Regla reflexiva	Evitar celdas ya marcadas como obstáculo
LangGraph	Command , reducers, state_schema , dynamic prompt

8. Goal-Based Reflex Agent — estado + simulación hacia una meta

La página 13 agrega una nueva capa sobre el modelo interno:

```
State = Modelo interno del mundo
        y cómo cambia sobre acciones

+ Simulación de estados basados en la meta
  que se busca lograr y así tomar una decisión

Robotics, Self Driving Cars
```

El diagrama añade:

Bloque nuevo	Rol
Goals	Define el estado deseado
Simulation of Action A to State X	Evalúa qué pasaría si se toma una acción

8.1 Qué lo distingue

Un Goal-Based Agent no se pregunta solo:

¿Qué acción corresponde a esta condición?

Se pregunta:

¿Qué secuencia de acciones me acerca al objetivo?

La decisión depende de una meta explícita. Si hay varias acciones posibles, se elige la que conduce a cumplir la meta, aunque no sea la reacción más obvia al percepto inmediato.

8.2 Formalización

Sea:

- $\hat{s}_t$ = estado interno actual,
- G = conjunto de estados meta,
- A = acciones disponibles,
- $T(s, a)$ = modelo de transición,
- $c(a)$ = costo de acción.

El agente busca una secuencia:

$$\pi^* = \arg \min_{\pi} \sum_{a \in \pi} c(a)$$

sujeta a:

$$T(\hat{s}_t, \pi) \in G$$

En lenguaje práctico: encuentra el plan de menor costo que llega al objetivo.

8.3 Algoritmos clásicos asociados

Algoritmo	Uso
BFS (<i>Breadth-First Search</i>)	Ruta más corta en grafos no ponderados
DFS (<i>Depth-First Search</i>)	Exploración profunda, útil pero no garantiza ruta óptima
Dijkstra	Ruta mínima con costos positivos
A*	Ruta mínima con heurística
STRIPS	Planificación simbólica clásica con precondiciones y efectos
Replanning	Recalcular plan cuando el entorno cambia

8.4 Limitación clave

Un Goal-Based Agent puede cumplir la meta, pero no necesariamente elige la **mejor** forma de cumplirla si no hay función de utilidad. Si el objetivo es "llegar al destino", cualquier ruta válida puede ser aceptable. Pero si además importan seguridad, costo, energía, tiempo y riesgo, hace falta un Utility-Based Agent.

9. Implementación del laboratorio — 03_goal_based_agent.py

El laboratorio implementa un robot de *picking* en almacén:

```
03_goal_based_agent.py
```

9.1 Caso implementado

El usuario entrega una orden de compra; el agente debe:

- 1. Encontrar el producto.
- 2. Planificar ruta al estante.
- 3. Confirmar recogida.
- 4. Informar si no hay stock suficiente.

9.2 Entorno

El script define:

Elemento	Implementación
Mapa del almacén	WAREHOUSE_GRID
Estantes/productos	SHELVES
Posición inicial	POSICION_INICIAL = (0, 0)
Planificador	_bfs_shortest_path

Productos del ejemplo:

Estante	Producto	Stock	Posición
E-12	auriculares bluetooth	40	(1, 5)
E-27	cargador USB-C	12	(5, 2)
E-33	mouse inalámbrico	3	(7, 6)

Estante	Producto	Stock	Posición
E-41	teclado mecánico	0	(3, 7)

9.3 Tools

Tool	Función
<code>buscar_producto(nombre_producto)</code>	Busca coincidencias por nombre
<code>planificar_ruta_a_estante(codigo_estante)</code>	Ejecuta BFS hacia el estante
<code>confirmar_recogida(codigo_estante, cantidad)</code>	Valida stock y descuenta unidades

9.4 La "razón" no está solo en el LLM

El script comenta que BFS es la "razón interna" del agente para planificar una ruta óptima. Esto es importante: en un sistema robusto, el LLM no debería inventar rutas paso a paso si existe un algoritmo determinista correcto.

Arquitectónicamente:

```

LLM:
    interpreta la orden, decide qué tool llamar y en qué orden

BFS:
    calcula la ruta correcta

Tool de stock:
    valida constraints de negocio

```

9.5 System prompt orientado a meta

El prompt explicita el objetivo:

```
Tu OBJETIVO explícito es cumplir la orden de compra que te da el usuario.
```

Y define el plan:

1. Localiza el producto.
2. Planifica el camino más corto.
3. Ejecuta la recogida.
4. Si no hay stock suficiente, informa claramente.

9.6 Observación crítica

Este ejemplo es técnicamente un híbrido:

- El **plan macro** está sugerido por el prompt.
- El **orden de llamadas** lo ejecuta el agente.
- La **ruta óptima** la calcula BFS.

Por eso, si aplicamos la clasificación de la Sesión 8, puede verse como un *workflow agéntico controlado* o un agente de baja autonomía: el LLM decide llamadas dentro de un plan bastante determinado.

10. Utility-Based Reflex Agent — maximizar utilidad, no solo cumplir metas

La página 14 extiende el Goal-Based Agent:

State = Modelo interno del mundo
y cómo cambia sobre acciones

+ Simulación de estados basados en la meta
que se busca lograr y así tomar una decisión

+ Establecer un mecanismo de recompensa hacia
un estado future (rank)

Drone Delivery, modelos de optimización
Maximiza su utilidad

El diagrama añade:

Bloque nuevo	Rol
Utility	Criterio numérico de preferencia
How Happy will I be with State X	Evaluación de deseabilidad del estado futuro

10.1 Diferencia con Goal-Based

Pregunta	Tipo de agente
¿Llego al objetivo?	Goal-Based
¿Cuál alternativa maximiza el valor esperado?	Utility-Based

Ejemplo:

- Goal-Based: "Entrega el paquete".
- Utility-Based: "Entrega el paquete maximizando una combinación de puntualidad, seguridad, costo y consumo energético".

10.2 Formalización

Un agente basado en utilidad elige:

$$a^* = \arg \max_{a \in A} \mathbb{E}[U(s') \mid \hat{s}_t, a]$$

donde:

- a^* = mejor acción,
- $U(s')$ = utilidad del estado resultante,
- $\hat{s}_t$ = estado estimado actual,
- s' = estado futuro posible.

Si hay múltiples atributos:

$$U(x) = \sum_{i=1}^n w_i \cdot u_i(x)$$

donde cada u_i mide un atributo (tiempo, costo, seguridad, energía) y w_i indica su peso relativo.

10.3 Qué resuelve

El agente basado en utilidad es necesario cuando:

- Hay varios objetivos en tensión.
- No basta con éxito/fracaso binario.
- Hay incertidumbre sobre resultados.
- Hay preferencias del usuario o del negocio.
- Se necesita justificar trade-offs.

10.4 Riesgo de diseño

La función de utilidad se vuelve una forma de gobernanza. Si está mal definida, el agente puede optimizar exactamente lo que se pidió, pero no lo que realmente se quería.

Ejemplos:

Mala utilidad	Resultado probable
Maximizar velocidad sin penalizar seguridad	Rutas peligrosas
Minimizar costo sin medir experiencia de usuario	Servicio barato pero deficiente
Maximizar conversiones sin controles	Recomendaciones invasivas
Penalizar errores visibles, no errores ocultos	El agente oculta incertidumbre

Principio de implementación: la utilidad debe incluir restricciones duras (*hard constraints*) y no solo preferencias blandas. Si una ruta es insegura o una acción viola política, su utilidad no debe ser "baja"; debe ser inadmisible.

11. Implementación del laboratorio — 04_utility_based_agent.py

El laboratorio implementa un planificador de despacho:

```
04_utility_based_agent.py
```

11.1 Caso implementado

El agente debe elegir vehículo y ruta maximizando utilidad combinada.

Vehículos:

Vehículo	Costo/km	Velocidad	Seguridad
furgon_1	0.9	60 km/h	0.95
moto_1	0.4	45 km/h	0.80
van_electrica_1	0.6	50 km/h	0.97

Rutas:

Ruta	Km	Tráfico
ruta_norte	18	0.7
ruta_centro	9	0.9
ruta_perimetral	26	0.2

11.2 Función de utilidad

El código calcula:

```

tiempo_h = r["km"] / v["vel_kmh"] * (1 + r["trafico"])
costo = r["km"] * v["costo_km"]

score_tiempo = max(0.0, 1 - tiempo_h / 2.0)
score_costo = max(0.0, 1 - costo / 30.0)
score_seguridad = v["seguridad"]

utilidad = (
    peso_tiempo * score_tiempo
    + peso_costo * score_costo
    + peso_seguridad * score_seguridad
)

```

Pesos por defecto:

Criterio	Peso
Tiempo	0.40
Costo	0.35
Seguridad	0.25

11.3 Tools

Tool	Función
listar_opciones_envio()	Lista vehículos y rutas disponibles
evaluar_opcion_envio(...)	Calcula utilidad de una combinación vehículo+ruta

11.4 System prompt

El prompt exige evaluar varias combinaciones:

1. Llama a ``listar_opciones_envio``.
 2. Evalúa AL MENOS 3 combinaciones distintas.
 3. Compara las utilidades.
 4. Recomienda la combinación con mayor utilidad explicando el trade-off.

Además permite adaptar pesos:

Si el usuario da preferencias ("prioriza seguridad"), ajusta los pesos.

11.5 Lectura de diseño

Este script muestra una buena separación de responsabilidades:

Responsabilidad	Dónde vive
Interpretar preferencia del usuario	LLM
Listar alternativas	Tool
Calcular utilidad	Código determinista
Comparar resultados	LLM + datos de tools
Explicar trade-off	LLM

La parte matemática queda fuera del LLM, lo que reduce alucinación y hace auditable la decisión.

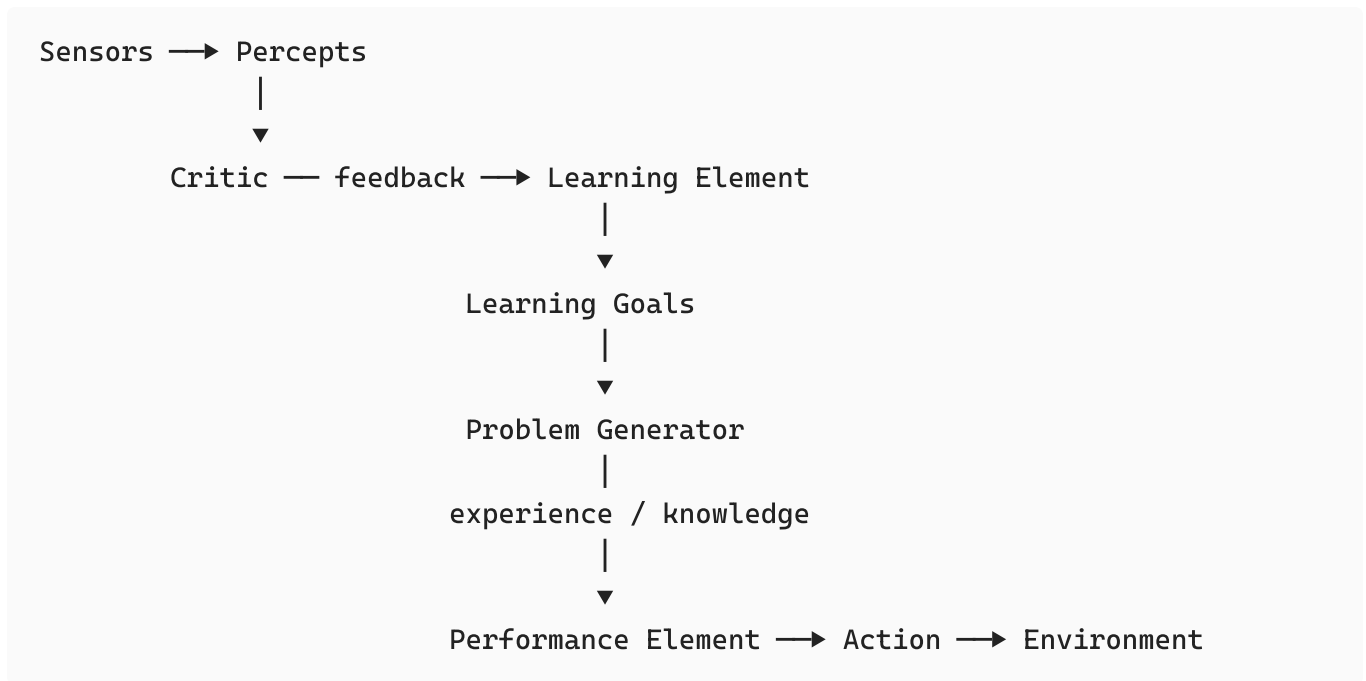
12. Learning Reflex Agent — aprendizaje por experiencia y feedback

La página 15 define el Learning Reflex Agent:

Aprende por experiencia basado en el feedback del ambiente.

El critic observa el resultado de las acciones en el ambiente y las compara con un standard de performance entregando un feedback numérico (RLEF).

El diagrama introduce la arquitectura clásica del agente de aprendizaje:



Componentes:

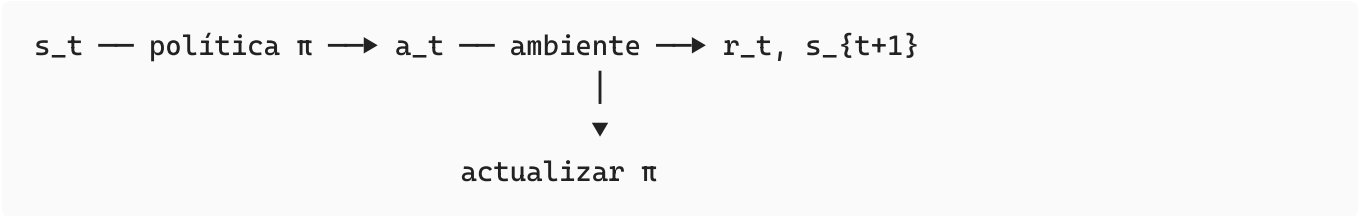
Componente	Función
Performance Element	Decide acciones con el conocimiento actual
Learning Element	Ajusta el conocimiento o la política
Critic	Evalúa resultados frente a un estándar de desempeño
Problem Generator	Propone exploraciones para descubrir mejores acciones
Experience	Evidencia acumulada por interacción con el ambiente

12.1 El loop de Reinforcement Learning según la página 16

La página 16 lista el ciclo:

Paso	Descripción
State observation	El agente percibe el estado actual del ambiente
Action selection	Según su política actual, escoge una acción
Environment feedback	El ambiente reacciona y entrega reward/penalty + nuevo estado
Policy update	El agente usa el feedback para actualizar su política

En forma compacta:



12.2 Sobre la sigla RLEF

El material usa RLEF. No es una sigla tan estandarizada como RL (*Reinforcement Learning*) o RLHF (*Reinforcement Learning from Human Feedback*). En el contexto de la diapositiva, la interpretación más coherente es:

RLEF = aprendizaje por refuerzo con feedback del environment/entorno

Conviene distinguir:

Sigla	Fuente de feedback	Uso típico
RL	Ambiente	Aprender políticas de acción por reward/penalty
RLHF	Humanos	Alinear modelos con preferencias humanas
Reflexion	Ambiente/tests/modelo → reflexión verbal	Mejorar agentes LLM sin actualizar pesos
Self-Refine	El propio LLM genera feedback y refina	Mejorar outputs en test-time

12.3 Formalización RL mínima

En un MDP (*Markov Decision Process*), el agente interactúa con:

$$\mathcal{M} = \langle S, A, P, R, \gamma \rangle$$

donde:

- S = estados,
- A = acciones,
- $P(s' \mid s, a)$ = dinámica de transición,
- $R(s, a, s')$ = recompensa,
- γ = factor de descuento.

El objetivo es aprender una política $\pi(a \mid s)$ que maximice retorno esperado:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

12.4 Learning Agent vs. memoria

Un Learning Agent no solo recuerda. **Usa lo recordado para cambiar su comportamiento futuro.**

Sistema	¿Recuerda?	¿Cambia política?
Chat con historial	Sí	No necesariamente
Agente con Store de preferencias	Sí	Puede personalizar, pero no siempre aprende
Learning Agent	Sí	Sí, ajusta decisiones por feedback

La diferencia es crítica: persistir datos en una base no convierte automáticamente al sistema en Learning Agent. Debe haber un mecanismo explícito de actualización.

13. Implementación del laboratorio — 05_learning_agent.py

El laboratorio implementa un agente de reposición de inventario que aprende por feedback:

```
05_learning_agent.py
```

13.1 Caso implementado

El agente recomienda cantidades de reposición para estantes. Después recibe feedback del usuario/ambiente sobre si la reposición fue correcta o produjo problemas como sobre-stock.

13.2 Store persistente

El aprendizaje se guarda en:

```
05_learning_store.json
```

El archivo actual contiene:

```
{
  "E-33": {
    "aciertos": 0,
```

```

    "fallos": 2,
    "confianza": 0.0
  }
}

```

Esto significa que el agente ya "aprendió" que sus recomendaciones previas para E-33 fallaron dos veces, por lo que su confianza quedó en 0.0.

13.3 Funciones internas

Función	Rol
<code>_cargar_aprendizaje()</code>	Lee el JSON persistente
<code>_guardar_aprendizaje(data)</code>	Guarda el JSON
<code>_consultar_confianza(estante)</code>	Obtiene aciertos, fallos y confianza
<code>_registrar_resultado(estante, acierto)</code>	Actualiza aciertos/fallos y recalcula confianza

La fórmula de confianza es simple:

$$\text{confianza} = \frac{\text{aciertos}}{\text{aciertos} + \text{fallos}}$$

13.4 Tools

Tool	Componente de Learning Agent
<code>consultar_estado_estante</code>	Percepción + memoria aprendida
<code>recomendar_cantidad_reposicion</code>	Performance element
<code>registrar_feedback_reposicion</code>	Critic + learning element

13.5 Exploración conservadora

La tool `recomendar_cantidad_reposicion` ajusta la cantidad si la confianza es baja:

```

if conf < 0.4:
    ajustada = max(1, cantidad_sugerida // 2)
    return "se recomienda EXPLORAR con {ajustada} unidades..."

```

Esto modela una forma simple de exploración prudente: cuando el historial muestra fallos, el agente reduce el tamaño de la acción para disminuir riesgo.

13.6 Lectura crítica

Este script no implementa RL completo con gradientes, Q-learning o entrenamiento de pesos. Implementa **aprendizaje simbólico/persistente por feedback**:

```
feedback → actualizar contador → recalcular confianza → cambiar recomendación futura
```

Para el objetivo pedagógico de la sesión, eso es suficiente: muestra la diferencia entre recordar información y modificar decisiones por experiencia.

14. Tabla comparativa del PDF — fortalezas, limitaciones y latencia

La página 18 presenta una tabla comparativa. Reconstruida y normalizada:

Tipo de agente	Mecanismo principal	Memoria	Ejemplo	Fortalezas	Limitaciones
Simple Reflex Agent	Reglas predefinidas condición-acción	No	Termostato	Rápido, confiable en entornos predecibles	Sin memoria, repite errores en escenarios dinámicos
Model-Based Reflex Agent	Estado interno + reglas condición-acción	Sí	Aspiradora robótica	Recuerda estados pasados, infiere partes no visibles del entorno	Sigue reaccionando a planificaciones
Goal-Based Agent	Simula futuros resultados para alcanzar metas	Sí	Auto autónomo	Planifica acciones, se adapta a condiciones cambiantes	No evalúa calidad de resultados solo si se cumple objetivo
Utility-Based Agent	Evalúa deseabilidad mediante función de utilidad	Sí	Dron de entrega autónomo	Optimiza resultado: velocidad, etc.	Requiere función de utilidad bien definida

Tipo de agente	Mecanismo principal	Memoria	Ejemplo	Fortalezas	Limitaciones
				seguridad, energía	precisión, decisiones más complejas
Learning Agent	Aprende por retroalimentación/recompensas	Sí	Bot de ajedrez con IA	Más adaptable; mejora continuamente	Requiere muchos datos para rendimiento óptimo

Leyenda de latencia de la diapositiva:

Nivel	Significado
Muy baja	Respuesta inmediata, casi sin procesamiento
Media	Requiere simulación o evaluación antes de actuar
Alta	Necesita entrenamiento, retroalimentación y tiempo para mejorar

14.1 Lectura transversal

La progresión aumenta capacidades, pero también aumenta costo:

Más simple	—————→				Más sofisticado
Reglas	Estado	Metas	Utilidad	Aprendizaje	
Barato	Memoria	Plan	Optimización	Adaptación	
Auditable	Parcial	Search	Trade-offs	Feedback	

No siempre conviene subir de nivel. El diseño correcto es el nivel mínimo que resuelve el problema con el riesgo aceptable.

15. Reasoning models — quién los necesita y quién no

La página 19 pregunta:

¿Qué agentes usan o podrían usar reasoning models?

Reconstrucción de la tabla:

Tipo de agente	¿Usa razonamiento profundo?	¿Podría beneficiarse de uno?	Justificación
Agente de Reflexión Simple	No	Poco útil	Solo ejecuta reglas fijas sin contexto ni inferencia
Agente de Reflexión con Modelo	No	Posible	Tiene estado interno, pero no simula ni planifica; un reasoning model podría mejorar inferencias
Agente Basado en Objetivos	Parcialmente	Sí	Simula futuros estados para alcanzar metas; reasoning puede mejorar planificación
Agente Basado en Utilidad	Parcialmente	Sí	Evalúa múltiples opciones; reasoning puede optimizar decisiones complejas
Agente de Aprendizaje	Sí	Sí	Aprende de experiencia y ajusta estrategias; reasoning puede enriquecer el aprendizaje

15.1 Interpretación práctica

Un *reasoning model* aporta más valor cuando el agente necesita:

- descomponer objetivos,
- planificar secuencias,
- comparar alternativas,
- manejar restricciones conflictivas,
- explicar trade-offs,
- aprender de feedback no estructurado,
- revisar errores propios.

En cambio, aporta poco cuando el comportamiento correcto es una regla fija y auditable.

15.2 Costo de usar reasoning

Beneficio	Costo
Mejor planificación	Mayor latencia
Mejor análisis de trade-offs	Más tokens

Beneficio	Costo
Mejor interpretación de contexto	Menor determinismo
Mejor recuperación ante errores	Necesidad de evals más fuertes

Regla de diseño: no pongas un modelo de razonamiento a decidir una condición que se puede evaluar con un `if`. Úsalo para lo que el código rígido no sabe representar bien: ambigüedad, planificación abierta, preferencias, explicación y recuperación.

16. Conexión con Sesión 8 — dimensiones funcionales del agente

La Sesión 8 definió:

Comunicación → Contexto → Entorno → Autonomía → Criticidad

Los tipos de agente de la Sesión 10 se insertan dentro de la capa de **Autonomía** y el modo de decisión dentro del **Entorno**:

Capa Sesión 8	Cómo se especializa en Sesión 10
Communication Layer	Puede ser conversacional o no; los agentes de control pueden operar vía API/sensores
Context Definition	Define rol, meta y criterios de decisión
Environment Definition	Sensores/perceptos, tools/actuadores, estado interno, knowledge base
Autonomy Dimension	El tipo de agente determina cuánto decide por sí mismo
Criticality Dimension	A mayor autonomía/utilidad/aprendizaje, más guardrails/evals/HITL necesita

16.1 Tipos de agente vs. autonomía

Tipo	Autonomía típica
Simple Reflex	Constrained
Model-Based Reflex	Constrained o Semi-Autonomous
Goal-Based	Semi-Autonomous

Tipo	Autonomía típica
Utility-Based	Semi-Autonomous, a veces Fully Autonomous en dominios acotados
Learning	Semi/Fully Autonomous solo con evaluación y controles fuertes

16.2 Tipos de agente vs. criticidad

La criticidad no depende solo del tipo, sino de la acción. Pero la tendencia es:

Simple Reflex < Model-Based < Goal-Based < Utility-Based < Learning

porque cada nivel agrega grados de libertad:

- más estado que puede estar equivocado,
- más pasos intermedios,
- más criterios de optimización,
- más cambios de comportamiento con el tiempo.

17. Conexión con Sesión 9 — memoria contextual y estado interno

La Sesión 9 separó:

Memoria	Rol
Short-term / Checkpointer	Estado de una conversación o ejecución
Long-term / Store	Información persistente entre sesiones
Episódica	Interacciones pasadas
Semántica	Knowledge base
Procedimental	Prompts, tools, guardrails

La Sesión 10 usa esas memorias así:

Tipo de agente	Memoria mínima necesaria
Simple Reflex	Ninguna; solo percepto actual
Model-Based Reflex	Short-term state o modelo interno
Goal-Based	Estado + representación de meta + posible mapa/modelo del entorno

Tipo de agente	Memoria mínima necesaria
Utility-Based	Estado + criterios/pesos + evaluaciones de alternativas
Learning	Store persistente + feedback histórico + política/confianza actualizada

17.1 El estado interno no siempre es conversación

En agentes LLM se tiende a pensar que memoria = historial de chat. Esta sesión corrige eso:

Caso	Memoria relevante
Robot explorador	Mapa interno
Despacho	Evaluaciones de rutas
Reposición	Confianza por estante
Asistente legal	Hechos del caso
Agente financiero	Perfil de riesgo, restricciones, posiciones

El historial conversacional puede ser útil, pero el agente necesita **estado de dominio**.

18. Investigación complementaria — ReAct como puente entre LLM y agente reflexivo

El patrón ReAct (*Reasoning and Acting*) conecta directamente con esta sesión porque alterna:

razonamiento → acción/tool → observación → razonamiento → acción/tool

Eso se parece al ciclo:

Sense → Plan/Decide → Act → Observe

18.1 Cómo se mapea ReAct a los tipos de la sesión

Tipo de agente	ReAct aporta
Simple Reflex	Poco; el razonamiento es innecesario si basta una regla
Model-Based Reflex	Observaciones de tools actualizan el estado
Goal-Based	Razonamiento ayuda a descomponer la meta

Tipo de agente	ReAct aporta
Utility-Based	Razonamiento ayuda a comparar alternativas
Learning	Observación/feedback puede alimentar memoria o reflexión

18.2 Riesgo

ReAct no garantiza planificación óptima. Solo estructura el bucle de razonamiento y acción. Si la acción requiere una ruta óptima, una función de utilidad o una política aprendida, esas piezas deben estar implementadas explícitamente como tools, evaluadores o memoria.

19. Investigación complementaria — Reflexion, Self-Refine y agentes que se critican

Aunque el curso usa "reflex agents" en el sentido clásico, en literatura moderna aparece **Reflexion**:

Intento → feedback → reflexión verbal → memoria episódica → nuevo intento

La idea de Reflexion es reforzar un agente de lenguaje **sin actualizar pesos**: el agente escribe reflexiones sobre por qué falló y las guarda en memoria para mejorar intentos posteriores.

19.1 Relación con Learning Agent

Learning Agent clásico	Reflexion
Feedback numérico o ambiental	Feedback textual, tests, evaluador o entorno
Actualiza política/modelo	Actualiza memoria verbal
Puede requerir muchos datos	Puede mejorar en pocos intentos
Aprendizaje en pesos o tabla/política	Aprendizaje en contexto/memoria

19.2 Self-Refine

Self-Refine propone:

Generar output inicial → generar feedback sobre ese output → refinar → repetir

A diferencia de Reflexion, no necesariamente hay ambiente externo ni acción física/digital; puede ser solo mejora iterativa de una respuesta.

19.3 Cuándo usarlos en proyectos

Patrón	Útil cuando
Reflexion	Hay tareas repetibles con feedback externo: tests, validadores, usuarios, métricas
Self-Refine	Hay outputs textuales/código/documentos que pueden mejorarse por crítica
LLM-as-a-Judge	Se necesita un critic separado que evalúe calidad
Tree of Thoughts	Hay búsqueda entre múltiples rutas de razonamiento

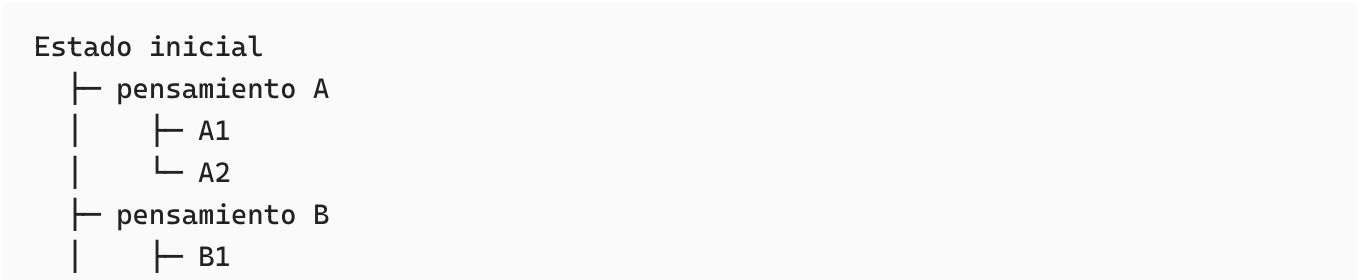
19.4 Advertencia

La auto-crítica de un LLM no es garantía de verdad. Si el mismo modelo genera, critica y corrige, puede compartir el mismo sesgo o punto ciego. Para tareas críticas, el critic debe apoyarse en:

- tests deterministas,
- datos externos,
- constraints formales,
- evaluadores independientes,
- revisión humana.

20. Investigación complementaria — Tree of Thoughts y búsqueda deliberada

Tree of Thoughts (ToT) generaliza Chain-of-Thought al permitir explorar varias rutas de razonamiento:



```
|   └─ B2
└─ pensamiento C
```

El modelo puede evaluar, mirar hacia adelante y retroceder. Esto es especialmente relevante para:

- Goal-Based Agents, porque mejora la búsqueda de planes.
- Utility-Based Agents, porque permite comparar rutas con criterios.
- Learning Agents, porque puede explorar estrategias alternativas.

Conexión con la sesión: la página 13 habla de simulación de estados futuros; ToT es una estrategia moderna de inferencia para simular y evaluar múltiples caminos de pensamiento antes de decidir.

21. El repositorio de laboratorio — mapa completo

La carpeta de la sesión contiene:

```
agents26_m4s10-main/
  requirements.txt
  README.md
  00_basic_agent.py
  01_simple_reflex_agent.py
  02_model_based_reflex_agent.py
  03_goal_based_agent.py
  04_utility_based_agent.py
  05_learning_agent.py
  05_learning_store.json
  *.ipynb
```

21.1 Dependencias

`requirements.txt` declara:

```
langchain>=1.3.0
langgraph>=1.2.0
langchain-ollama>=1.1.0
ipykernel>=6.29.0
jupyter>=1.0.0
```

El README indica que los ejemplos usan:

- LangChain 1.x,
- LangGraph 1.x,
- `create_agent` ,
- Ollama local,
- modelo `llama3.2` .

21.2 `00_basic_agent.py`

Este archivo es el mínimo agente de prueba:

Elemento	Uso
<code>load_dotenv()</code>	Carga variables de entorno
<code>create_agent</code>	Construye agente
<code>@tool saludo</code>	Tool simple que saluda
Modelo	<code>anthropic:claude-opus-4-8</code> en el archivo
Prompt	Agente útil que controla robot 2D

Este archivo no forma parte de la taxonomía IBM; sirve como base mínima para entender `create_agent` .

21.3 Mapeo de scripts a tipos

Tipo	Script	Idea
Simple Reflex	<code>01_simple_reflex_agent.py</code>	Climatización por reglas
Model-Based Reflex	<code>02_model_based_reflex_agent.py</code>	Robot explorador con mapa interno
Goal-Based	<code>03_goal_based_agent.py</code>	Picking con búsqueda BFS
Utility-Based	<code>04_utility_based_agent.py</code>	Despacho con función de utilidad
Learning	<code>05_learning_agent.py</code>	Reposición con confianza aprendida

21.4 Patrón de implementación común

Todos los scripts siguen una estructura didáctica:

1. Definir entorno simulado
 2. Definir tools de percepción/acción/evaluación

- 3. Definir system prompt
- 4. Crear agente con create_agent
- 5. Exponer función reutilizable
- 6. Permitir ejecución directa con __main__

Esto coincide con la filosofía del curso: simular la arquitectura antes de depender de frameworks complejos.

22. LangChain/LangGraph — decisiones técnicas relevantes

La documentación actual de LangChain define un agente como:

modelo + tools + loop hasta completar tarea

create_agent construye ese loop sobre LangGraph.

22.1 Componentes usados por el laboratorio

Componente	Dónde aparece	Por qué importa
create_agent	Todos los scripts	Construye el runtime de agente
@tool	Todos los scripts	Expone funciones como acciones
system_prompt	Todos los scripts	Define rol, reglas y criterio de decisión
state_schema	02_model_based_reflex_agent.py	Añade estado interno
@dynamic_prompt	02_model_based_reflex_agent.py	Inyecta estado dinámico en el prompt
Command(update=...)	02_model_based_reflex_agent.py	Permite a una tool actualizar el estado
JSON local	05_learning_agent.py	Store persistente simple

22.2 Relación con checkpointer/store

El laboratorio no usa un checkpointer explícito en todos los scripts. Para producción, la decisión debería ser:

Necesidad	Mecanismo
Recordar mensajes dentro de una conversación	Checkpoint
Guardar estado interno de ejecución	State schema + checkpoint
Recordar datos entre sesiones	Store
Guardar aprendizaje histórico	Store persistente o base de datos

El `05_learning_store.json` es una versión mínima de Store. En producción debería reemplazarse por una base robusta: PostgreSQL, Redis, MongoDB, DynamoDB u otra según arquitectura.

22.3 Docstrings como parte del contexto

La Sesión 9 explicó que los docstrings de herramientas compiten por espacio en el Context Window. En esta sesión se ve en la práctica:

```
@tool
def evaluar_opcion_envio(...):
    """Calcula la utilidad (0-1, mayor es mejor) ..."""
```

El LLM decide cuándo y cómo usar una tool a partir de esa descripción. Por eso una mala descripción puede convertirse en mala decisión.

23. Cómo elegir el tipo de agente para un proyecto

23.1 Árbol de decisión práctico

```
¿La acción correcta depende solo del percepto actual?
├ Sí → Simple Reflex
└ No
    ¿Necesito recordar estado del entorno, pero no planificar?
    ├── Sí → Model-Based Reflex
    └ No
        ¿Tengo una meta clara de éxito/fracaso?
        ├── Sí → Goal-Based
        └ No
            ¿Debo balancear múltiples criterios?
            ├── Sí → Utility-Based
            └ No
                ¿El sistema debe mejorar con feedback histórico?
```

- └ Sí → Learning Agent
- └ Replantear el problema

23.2 Heurística de mínimo nivel suficiente

Si tu caso es...	Empieza con...
Umbrales, políticas simples, clasificación cerrada	Simple Reflex
Formularios, trámites, estados de proceso	Model-Based Reflex
Rutas, planificación de tareas, cumplimiento de orden	Goal-Based
Decisiones con trade-offs de negocio	Utility-Based
Personalización o mejora por feedback	Learning

23.3 Señales de que elegiste un nivel demasiado alto

- Usas LLM para evaluar condiciones triviales.
- No puedes explicar por qué el agente tomó una acción.
- El costo/latencia subió sin mejora observable.
- El agente "aprende" cosas que deberían ser políticas fijas.
- El sistema cambia comportamiento sin trazabilidad.

23.4 Señales de que elegiste un nivel demasiado bajo

- El agente repite errores porque no recuerda.
- El sistema falla cuando falta una observación directa.
- Hay demasiadas reglas ad hoc.
- Las reglas no capturan preferencias o trade-offs.
- El usuario espera adaptación y el sistema no mejora.

24. Plantilla — actualizar el Agent Profile Card con tipo reflexivo

La tarea de la sesión pide actualizar el *Profile Card* del proyecto con memoria y tipo(s) de agentes reflexivos. Plantilla recomendada:

ID AGENTE / SISTEMA AGÉNTICO
Nombre:
Contexto de negocio:

Dominio:

Usuario objetivo:

TIPO DE AGENTE REFLEXIVO

Tipo principal:

[Simple Reflex / Model-Based Reflex / Goal-Based / Utility-Based / Learning]

Justificación:

[Por qué este tipo es suficiente o necesario]

Perceptos:

[Qué observa el agente: input usuario, sensor, API, base de datos, evento]

Estado interno:

[Qué variables de estado mantiene; si no aplica, indicar "no mantiene estado"]

Reglas condición-acción:

[Reglas explícitas si aplica]

Modelo del mundo:

[Cómo representa el entorno y cómo se actualiza]

Metas:

[Estados objetivo o criterios de éxito]

Función de utilidad:

[Atributos, pesos, restricciones duras, fórmula]

Feedback y aprendizaje:

[Qué feedback recibe, quién lo entrega, cómo actualiza su comportamiento]

Memoria:

Corto plazo / Checkpointer:

Largo plazo / Store:

Episódica:

Semántica:

Procedimental:

Tools / Actuadores:

[Nombre, descripción, input, output, riesgo]

Autonomía:

[Constrained / Semi-Autonomous / Fully Autonomous]

Criticidad:

[Baja / Media / Alta + justificación]

Guardrails y HITL:

[Qué acciones requieren aprobación humana]

Evals:

[Cómo se medirá si decide bien]

Implementación Python:

Script propuesto:

Librerías:

Estado persistente:

Datos de prueba:

25. Ejemplo de actualización para un proyecto tipo VeterinarIA

Tomando el caso recurrente del curso:

Campo	Diseño sugerido
Nombre	VeterinarIA
Tipo principal	Model-Based Reflex + Goal-Based
Perceptos	Síntomas reportados, especie, raza, edad, historial
Estado interno	Perfil de mascota, síntomas actuales, preguntas respondidas
Meta	Orientar triage y decidir si requiere consulta veterinaria urgente
Utilidad	Podría añadirse para balancear urgencia, costo y riesgo clínico
Learning	Solo con mucha cautela; feedback clínico debería venir de expertos
Autonomía	Constrained
Criticidad	Alta si afecta decisiones de salud
HITL	Derivar a veterinario ante signos de alarma

25.1 Decisiones recomendadas

- No usar Simple Reflex puro salvo para reglas de alarma: "si hay convulsiones, derivar urgente".
- Usar Model-Based para mantener síntomas ya preguntados y evitar repetir preguntas.
- Usar Goal-Based para completar un set mínimo de información antes de orientar.

- Evitar Learning autónomo con feedback de usuarios no experto para recomendaciones médicas.
- Usar Evals con casos clínicos diseñados por veterinarios.

26. Ejemplo de actualización para BurSee, asesor bursátil

Campo	Diseño sugerido
Nombre	BurSee
Tipo principal	Utility-Based + Model-Based
Perceptos	Portafolio, mercado, perfil de riesgo, consulta usuario
Estado interno	Posiciones, preferencias, horizonte, restricciones
Meta	Recomendar decisiones consistentes con estrategia del usuario
Utilidad	Retorno esperado, riesgo, liquidez, diversificación, costos
Learning	Aprender preferencias operativas, no prometer rentabilidad
Autonomía	Semi-Autonomous con confirmación antes de operar
Criticidad	Alta por impacto financiero
HITL	Obligatorio para compra/venta

26.1 Función de utilidad posible

$$U = w_r \cdot R_{esperado} - w_\sigma \cdot Riesgo - w_c \cdot Costo + w_d \cdot Diversificación + w_l \cdot Liquidez$$

Restricciones duras:

- No operar sin confirmación.
- No recomendar fuera del perfil de riesgo.
- No inventar datos de mercado.
- No usar datos personales sin autorización.
- Registrar explicación y evidencia de cada recomendación.

27. Riesgos y controles por tipo

Tipo	Riesgo principal	Control recomendado
Simple Reflex	Reglas incompletas	Tests de tabla de decisión
Model-Based Reflex	Estado interno divergente	Reconciliación con fuente de verdad
Goal-Based	Plan válido pero frágil	Replanning + simulaciones
Utility-Based	Función de utilidad mal especificada	Revisión de pesos, constraints, análisis de sensibilidad
Learning	Aprende conducta indeseada	Feedback validado, auditoría, rollback, límites de actualización

27.1 Controles mínimos en producción

- Logging de perceptos, acciones y estado.
- Versionado de prompts y tools.
- Tests deterministas para reglas.
- Evals para planificación y utilidad.
- Monitoreo de drift en Learning Agents.
- HITL para acciones irreversibles o críticas.
- Separación entre recomendación y ejecución.

28. Cómo evaluar cada tipo

Tipo	Eval principal
Simple Reflex	Cobertura de reglas condición-acción
Model-Based Reflex	Exactitud del estado interno tras secuencias de eventos
Goal-Based	Tasa de cumplimiento de metas y optimalidad de plan
Utility-Based	Selección de alternativa con mayor utilidad y explicación de trade-off
Learning	Mejora de desempeño con feedback y ausencia de regresiones

28.1 Métricas sugeridas

Métrica	Aplica a
Accuracy de acción	Simple, Model-Based

Métrica	Aplica a
State reconstruction accuracy	Model-Based
Plan success rate	Goal-Based
Path optimality / cost ratio	Goal-Based
Utility regret	Utility-Based
Reward promedio acumulado	Learning
Time-to-improvement	Learning
Tasa de intervención humana	Todos con HITL
Incidentes por acción	Todos en producción

28.2 Utility regret

Para agentes de utilidad:

$$\text{regret} = U(a^*) - U(a_{\text{elegida}})$$

Si el regret promedio crece, el agente está eligiendo alternativas subóptimas aunque parezcan razonables en lenguaje natural.

29. Errores comunes al implementar estos agentes con LLMs

Error	Consecuencia	Corrección
Pedir al LLM que calcule todo	Alucinación numérica	Pasar cálculos a tools deterministas
No persistir estado	El agente olvida contexto operativo	Usar <code>state_schema</code> , <code>checkpoint</code> o <code>Store</code>
Usar memoria como aprendizaje	El agente recuerda pero no mejora	Definir mecanismo de actualización de política
No separar constraints de preferencias	El agente "optimiza" violando reglas	Hard constraints antes de utilidad
Tools con docstrings ambiguos	Tool calls incorrectos	Descripciones precisas con inputs/outputs
Sin evals por trayectoria	Solo se evalúa respuesta final	Evaluar pasos intermedios y acciones
Sin HITL en acciones críticas	Riesgo operativo/legal	Aprobación humana antes de ejecutar

30. Síntesis — lo que hay que llevarse de esta sesión

1. **La Sesión 10 clasifica el mecanismo de decisión del agente**, no solo sus componentes. La pregunta central es: ¿cómo pasa de percepción a acción?
2. **Simple Reflex Agent** usa reglas condición-acción sobre el percepto actual. Es rápido y auditable, pero no tiene memoria ni aprendizaje.
3. **Model-Based Reflex Agent** añade estado interno: recuerda o infiere cómo está el mundo aunque no pueda observarlo completo.
4. **Goal-Based Agent** añade metas y simulación/búsqueda de estados futuros. Ya no reacciona solo al presente; planifica para alcanzar un objetivo.
5. **Utility-Based Agent** añade una función de utilidad. No solo pregunta si logra la meta, sino qué alternativa produce el mejor resultado según criterios ponderados.
6. **Learning Agent** añade feedback y actualización de comportamiento. No basta con guardar historial: debe usarlo para cambiar futuras decisiones.
7. **Reasoning models** aportan poco en reglas simples, pero pueden ser valiosos en planificación, comparación de alternativas, aprendizaje verbal y recuperación de errores.
8. **El laboratorio implementa la taxonomía completa con LangChain/Ollama**, usando tools, prompts, estado de LangGraph, BFS, funciones de utilidad y un Store JSON persistente.
9. **La memoria de Sesión 9 se vuelve estado operativo en Sesión 10**: mapa interno, preferencias, historial de feedback, pesos de utilidad y confianza aprendida.
10. **La arquitectura correcta suele ser híbrida**: reglas deterministas para constraints, algoritmos clásicos para búsqueda/cálculo, LLM para interpretación, coordinación y explicación.

31. Checklist práctico — actualizar tu proyecto

Tipo de agente:

- ☐ ¿Identifiqué si mi proyecto necesita Simple Reflex, Model-Based, Goal-Based, Utility-Based o Learning?
- ☐ ¿Justifiqué por qué ese nivel es suficiente?
- ☐ ¿Evité usar un nivel más complejo solo porque "suena más agéntico"?

Percepción y estado:

- ☐ ¿Definé exactamente qué perceptos recibe el agente?
- ☐ ¿Separé input conversacional de señales de entorno/API/sensores?

☐ ¿Definí qué estado interno mantiene y cómo se actualiza?

Reglas, metas y utilidad:

☐ Si hay reglas, ¿están escritas como tabla condición-acción?

☐ Si hay metas, ¿está definido el estado objetivo?

☐ Si hay utilidad, ¿hay fórmula, pesos y restricciones duras?

☐ ¿Probé sensibilidad de los pesos de utilidad?

Aprendizaje:

☐ ¿El agente realmente cambia comportamiento por feedback o solo guarda historial?

☐ ¿El feedback es confiable?

☐ ¿Hay forma de revertir aprendizaje incorrecto?

☐ ¿Hay auditoría de cambios en política/confianza?

Implementación:

☐ ¿Las tools tienen docstrings claros?

☐ ¿Los cálculos críticos están en código determinista?

☐ ¿El estado usa `state_schema`, `checkpoint` o `Store` según corresponda?

☐ ¿El learning store está persistido en una base adecuada para producción?

Riesgo:

☐ ¿Clasifiqué criticidad por acción?

☐ ¿Definí HITL para acciones críticas?

☐ ¿Tengo logs de `percepto` → `estado` → `acción` → `resultado`?

☐ ¿Tengo evals por trayectoria, no solo por respuesta final?

32. Quiz de la sesión (con respuestas)

#	Pregunta	Respuesta correcta
1	¿Qué caracteriza a un Simple Reflex Agent?	Usa reglas condición-acción sobre la percepción actual, sin memoria ni feedback
2	¿Qué agrega un Model-Based Reflex Agent frente a uno simple?	Un estado/modelo interno del mundo que se actualiza con percepciones y acciones
3	¿Qué distingue a un Goal-Based Agent?	Simula o planifica estados futuros para alcanzar una meta explícita

#	Pregunta	Respuesta correcta
4	¿Qué añade un Utility-Based Agent sobre un Goal-Based Agent?	Una función de utilidad que rankea alternativas y permite optimizar trade-offs
5	¿Cuáles son los cuatro componentes clásicos de un Learning Agent?	Performance element, Learning element, Critic y Problem generator
6	¿Por qué un reasoning model es poco útil para un agente de reflexión simple?	Porque solo ejecuta reglas fijas sin necesidad de inferencia profunda
7	En el laboratorio, ¿qué script usa <code>state_schema</code> y <code>dynamic_prompt</code> ?	<code>02_model_based_reflex_agent.py</code>
8	En el laboratorio, ¿qué representa <code>05_learning_store.json</code> ?	Memoria persistente de aprendizaje: aciertos, fallos y confianza por estante
9	¿Cuál es la diferencia entre guardar memoria y aprender?	Guardar memoria conserva datos; aprender cambia la política o comportamiento futuro usando feedback
10	¿Cuál es una buena práctica para Utility-Based Agents críticos?	Usar constraints duros antes de optimizar la utilidad ponderada

33. Referencias

Del material original y archivos locales:

- `SES10_M4_ReflexAgents.pdf` — Sesión 10, Módulo 4, UTEC Posgrado.
- `agents26_m4s10-main/README.md` — descripción del laboratorio con LangChain + Ollama.
- `agents26_m4s10-main/00_basic_agent.py` — agente mínimo con `create_agent`.
- `agents26_m4s10-main/01_simple_reflex_agent.py` — Simple Reflex Agent de climatización.
- `agents26_m4s10-main/02_model_based_reflex_agent.py` — Model-Based Reflex Agent con `AgentState`, `reducers`, `Command` y `dynamic_prompt`.
- `agents26_m4s10-main/03_goal_based_agent.py` — Goal-Based Agent con BFS para picking.
- `agents26_m4s10-main/04_utility_based_agent.py` — Utility-Based Agent con función multiatributo.
- `agents26_m4s10-main/05_learning_agent.py` — Learning Agent con feedback persistente.

- `agents26_m4s10-main/05_learning_store.json` — Store JSON de aprendizaje.

Investigación complementaria:

- Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach*. Capítulo 2: agentes inteligentes; estructura de agentes; simple reflex, model-based, goal-based, utility-based y learning agents. Sitio AIML: <https://aima.cs.berkeley.edu/>
- IBM Think — *Types of AI agents*. Clasificación práctica de los cinco tipos de agentes: <https://www.ibm.com/think/topics/ai-agent-types>
- IBM Think — *What is a Model-Based Reflex Agent?*: <https://www.ibm.com/think/topics/model-based-reflex-agent>
- IBM Think — *What is a Goal-Based Agent?*: <https://www.ibm.com/think/topics/goal-based-agent>
- IBM Think — *What is a Utility-Based Agent?*: <https://www.ibm.com/think/topics/utility-based-agent>
- Sutton, R. & Barto, A. — *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
- LangChain Docs — Agents / `create_agent` : <https://docs.langchain.com/oss/python/langchain/agents>
- LangChain Reference — `create_agent` : https://reference.langchain.com/python/langchain/agents/factory/create_agent
- LangGraph Docs — Persistence, checkpoints and stores: <https://docs.langchain.com/oss/python/langgraph/persistence>
- Yao, S. et al. — *ReAct: Synergizing Reasoning and Acting in Language Models* (2022): <https://arxiv.org/abs/2210.03629>
- Shinn, N. et al. — *Reflexion: Language Agents with Verbal Reinforcement Learning* (2023): <https://arxiv.org/abs/2303.11366>
- Madaan, A. et al. — *Self-Refine: Iterative Refinement with Self-Feedback* (2023): <https://arxiv.org/abs/2303.17651>
- Yao, S. et al. — *Tree of Thoughts: Deliberate Problem Solving with Large Language Models* (2023): <https://arxiv.org/abs/2305.10601>
- Chen, L. et al. — *Decision Transformer: Reinforcement Learning via Sequence Modeling* (2021): <https://arxiv.org/abs/2106.01345>
- OpenAI — *Aligning language models to follow instructions* / InstructGPT y RLHF (2022): <https://openai.com/index/instruction-following/>

agents26_m4s10-main , más investigación propia sobre taxonomía clásica de agentes, reinforcement learning, agentes LLM con reflexión y patrones modernos de implementación con LangChain/LangGraph. Última actualización: 2026-07-21.